Computer Programming 1

Functions (part 3)

(16-10-2026)

Outline

- (Quick) Recap *(part 1)*
  - Introduction (Functions, Library of functions, Procedures, …)
  - Definition, Declaration and Call of Functions
  - Function parameters and local variables
- Parameters passing *(part 2)*
- **Recursive functions *(part 3)***

Recursive functions

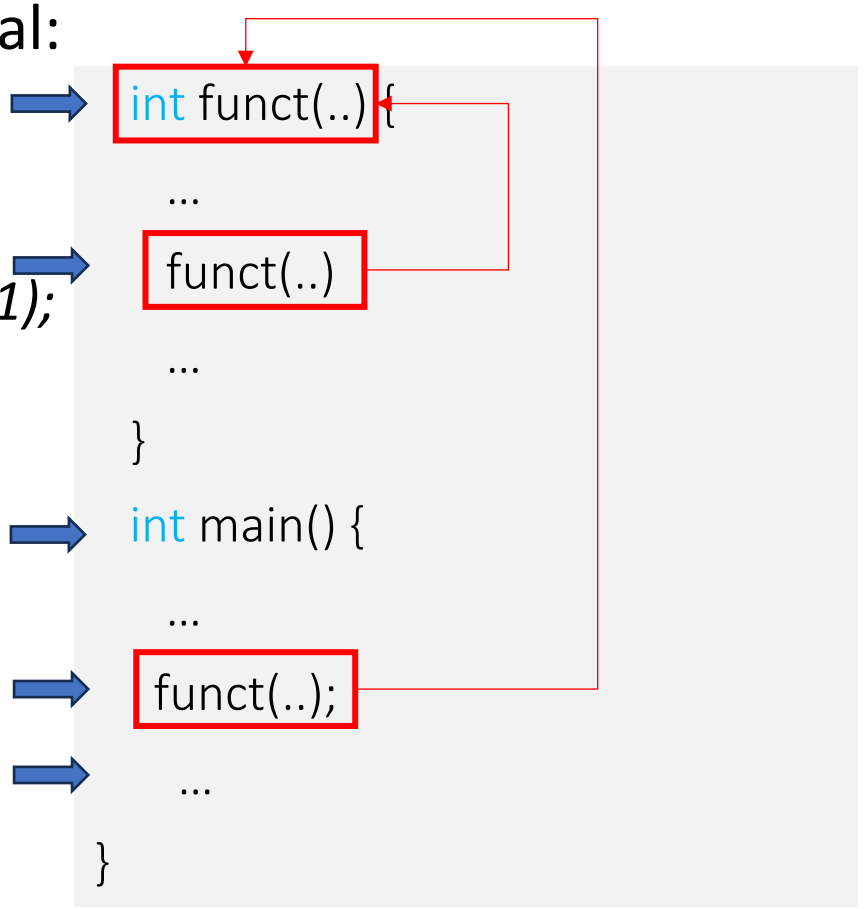
- A function that invokes itself is named "**recursive function**"
  - Note: two or more functions can also call each other (mutually recursive functions)
- Formulating some problems recursively is (often) natural:

  E.g.,
  - Factorial: $0! \stackrel{\text{def}}{=} 1;\ n! \stackrel{\text{def}}{=} n*(n-1)!$
  - Fibonacci: $F_0=1;\ F_1=1;\ F_n=F_{n-1}+F_{n-2}$
  - Even/odd numbers: $even(n) \Leftrightarrow odd(n-1);\ odd(n) \Leftrightarrow even(n-1);$
  - Expressions: $sum \stackrel{\text{def}}{=} number;\ sum \stackrel{\text{def}}{=} (sum + sum)$
- Two components:
  - one or more recursive calls
  - one or more ends conditions
- Attention:
  - Risk of producing infinite sequences (e.g., cycles)

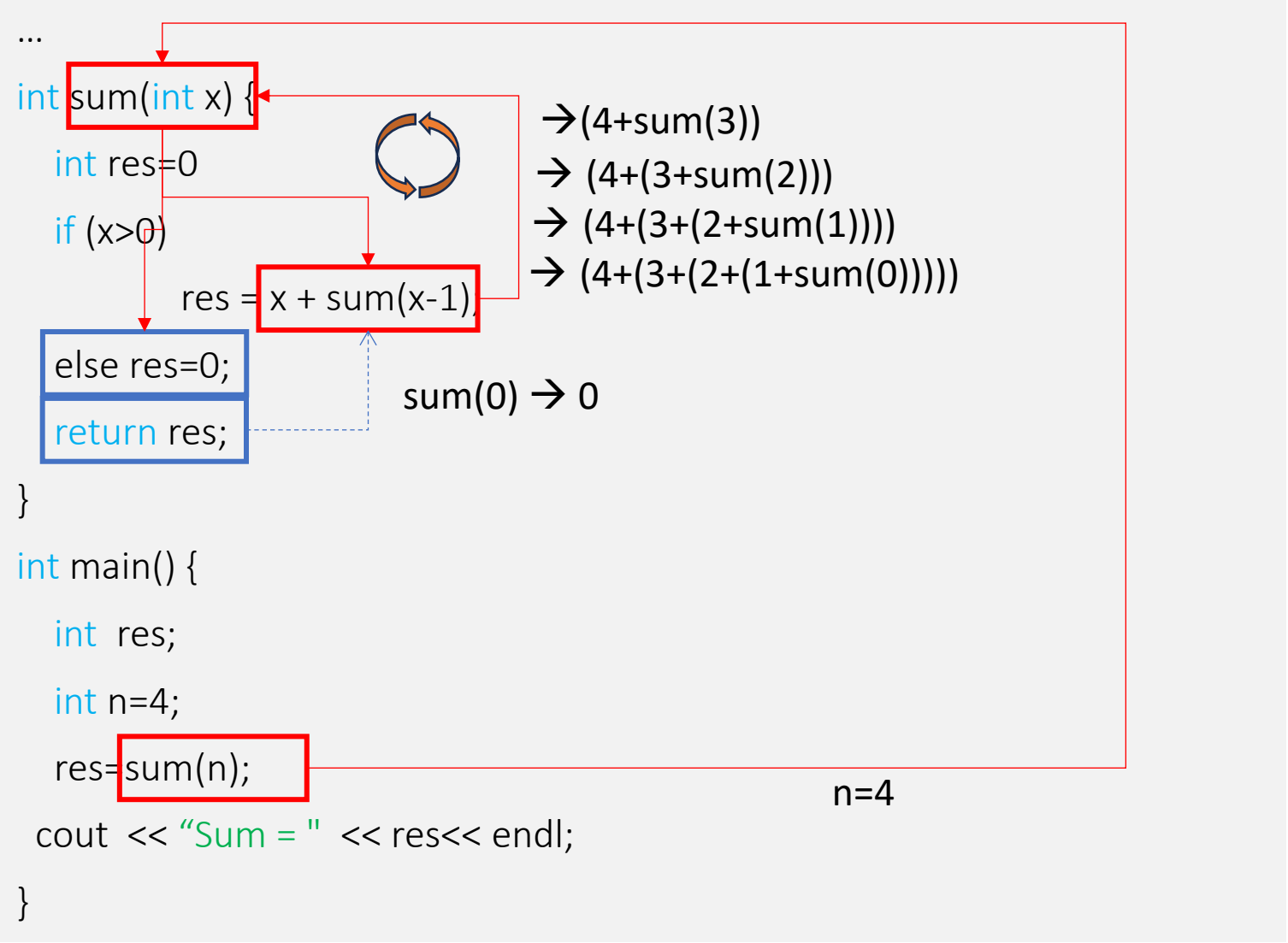
Recursive functions

- A function that invokes itself is named "**recursive function**"
  - Note: two or more functions can also call each other (mutually recursive functions)
- Formulating some problems recursively is (often) natural:

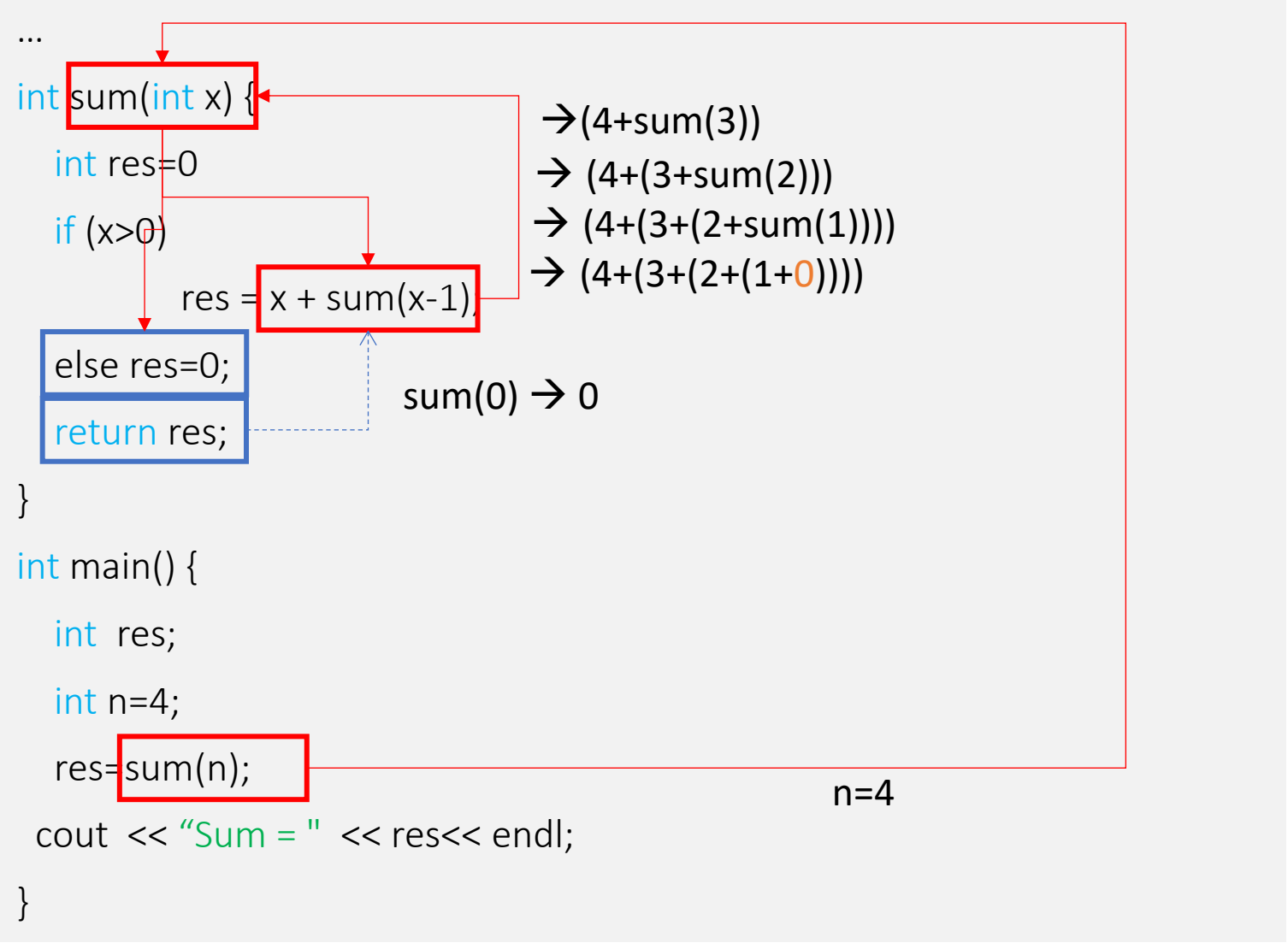
  E.g.,
  - Factorial: *0!* $\stackrel{\text{def}}{=}$ *1; n!* $\stackrel{\text{def}}{=}$ *n*(n-1)!*
  - Fibonacci: $F_0=1$; $F_1=1$; $F_n=F_{n-1}+F_{n-2}$
  - Even/odd numbers: *even(n)⇔ odd(n-1); odd(n) ⇔ even(n-1);*
  - Expressions: *sum* $\stackrel{\text{def}}{=}$ *number; sum* $\stackrel{\text{def}}{=}$ *(sum + sum)*
- Two components:
  - one or more recursive calls
  - one or more ends conditions
- Attention:
  - Risk of producing infinite sequences (e.g., cycles)



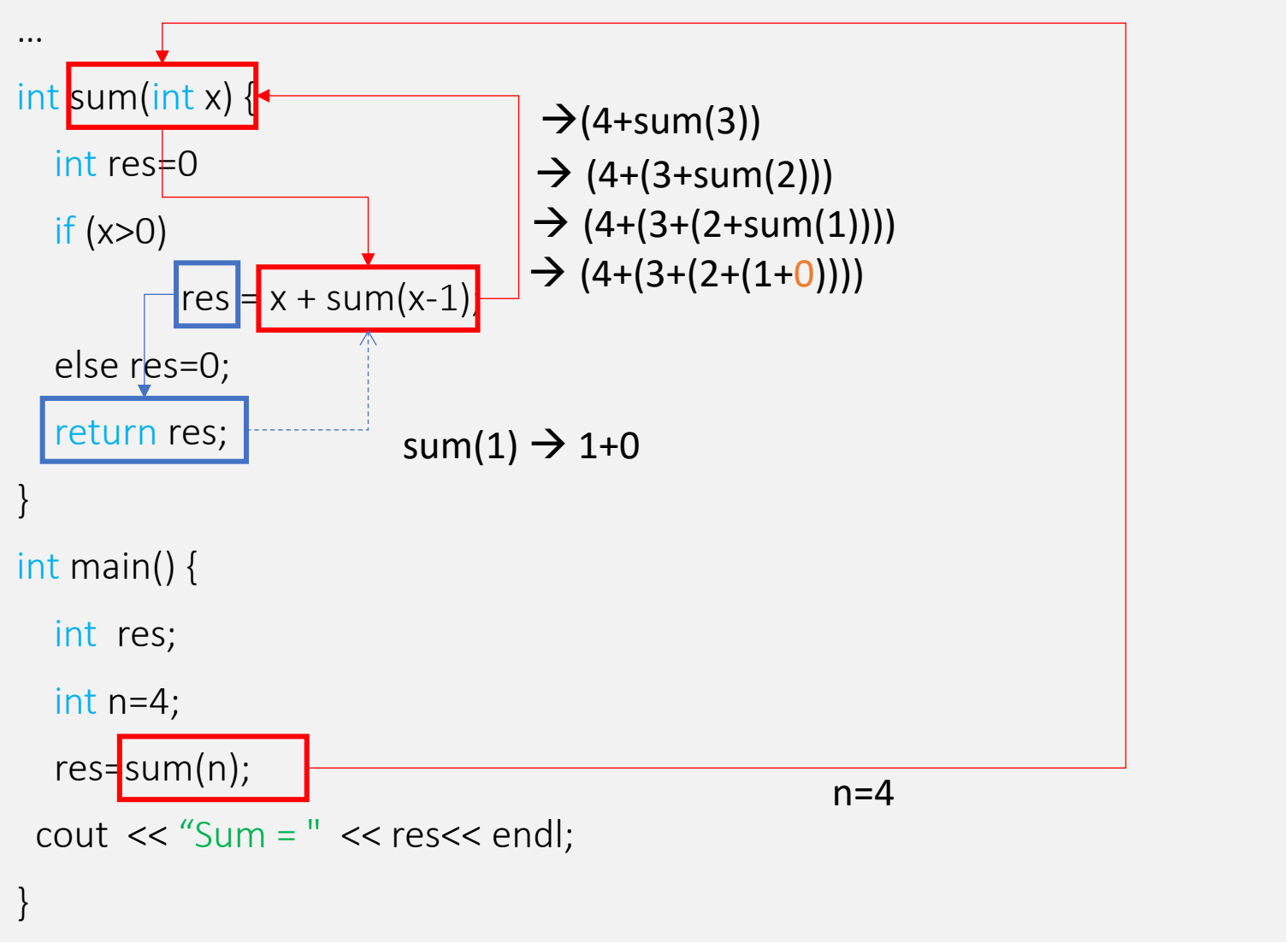
Example of Recursive function: sum



Example of Recursive function: sum



Example of Recursive function: sum



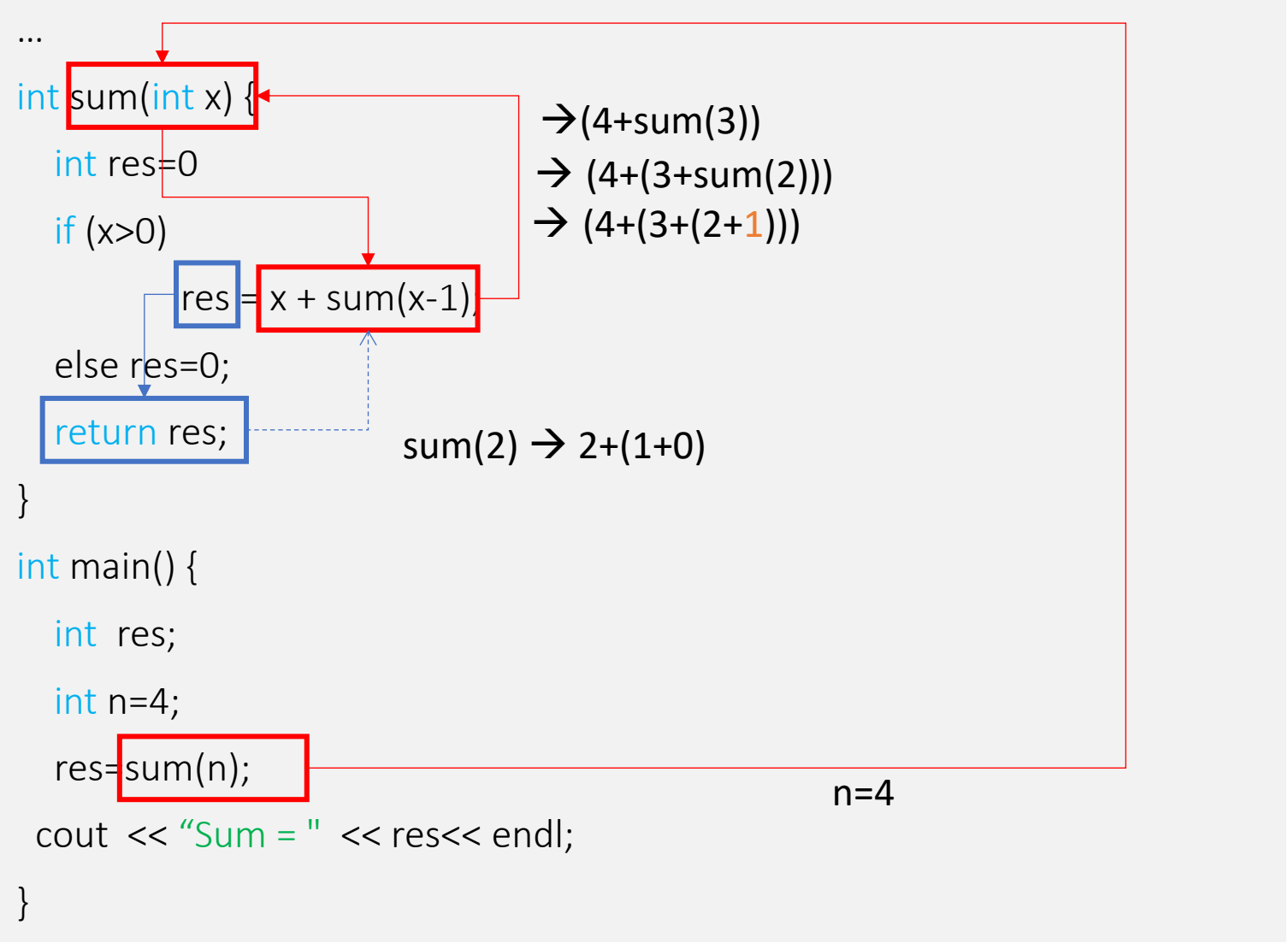
Example of Recursive function: sum

```
...
int sum(int x) {
    int res=0
    if (x>0)
            res = x + sum(x-1);
    else res=0;
    return res;
}
int main() {
    int  res;
    int n=4;
    res=sum(n);
  cout  << "Sum = "  << res<< endl;
}
```

→(4+sum(3))
→ (4+(3+sum(2)))
→ (4+(3+(2+1)))

n=4

Example of Recursive function: sum



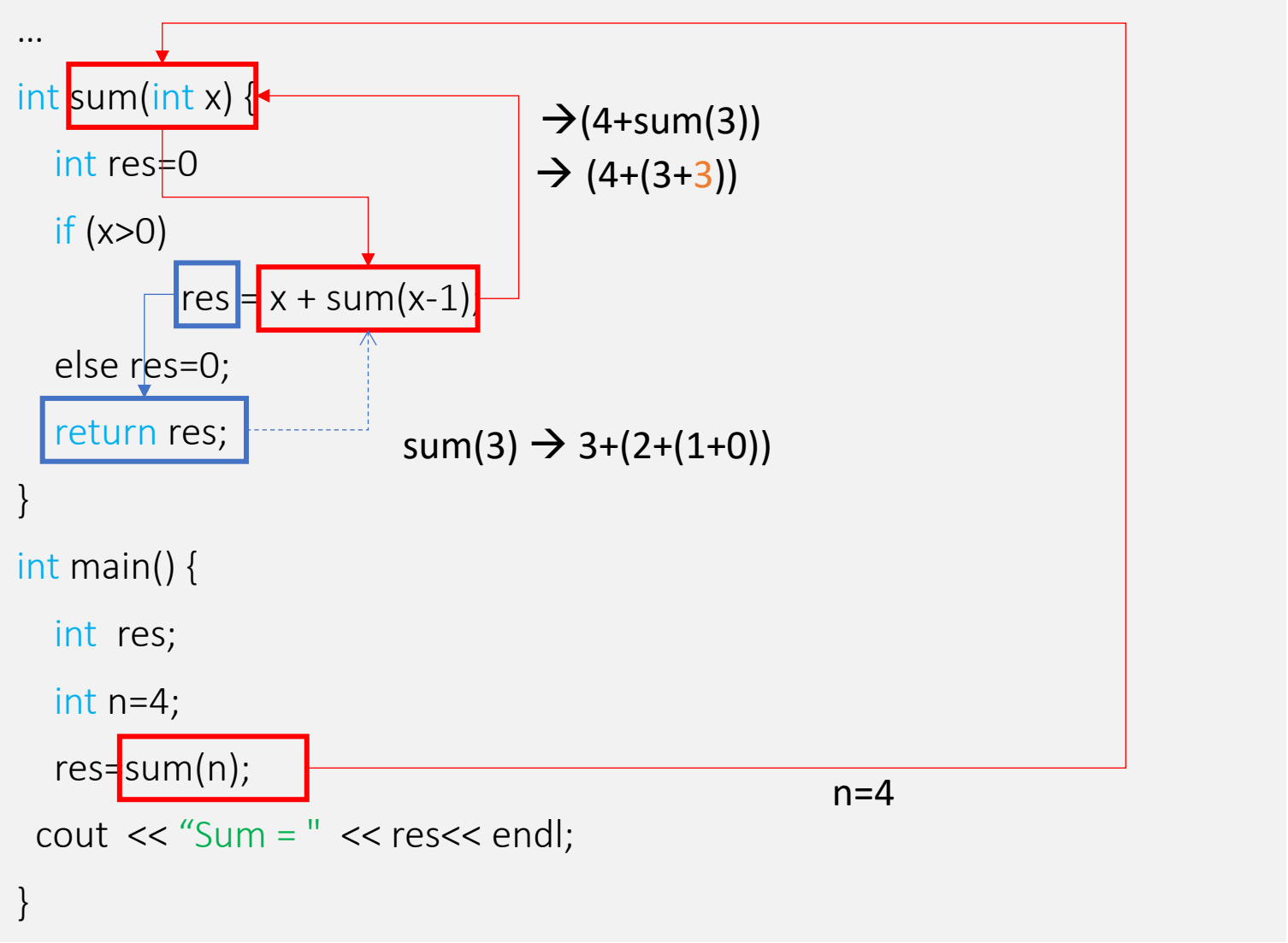
Example of Recursive function: sum

```
...
int sum(int x) {
    int res=0
    if (x>0)
            res = x + sum(x-1);
    else res=0;
    return res;
}
int main() {
    int  res;
    int n=4;
    res=sum(n);
  cout  << "Sum = "  << res<< endl;
}
```

→(4+sum(3))
→ (4+(3+3))

n=4

Example of Recursive function: sum



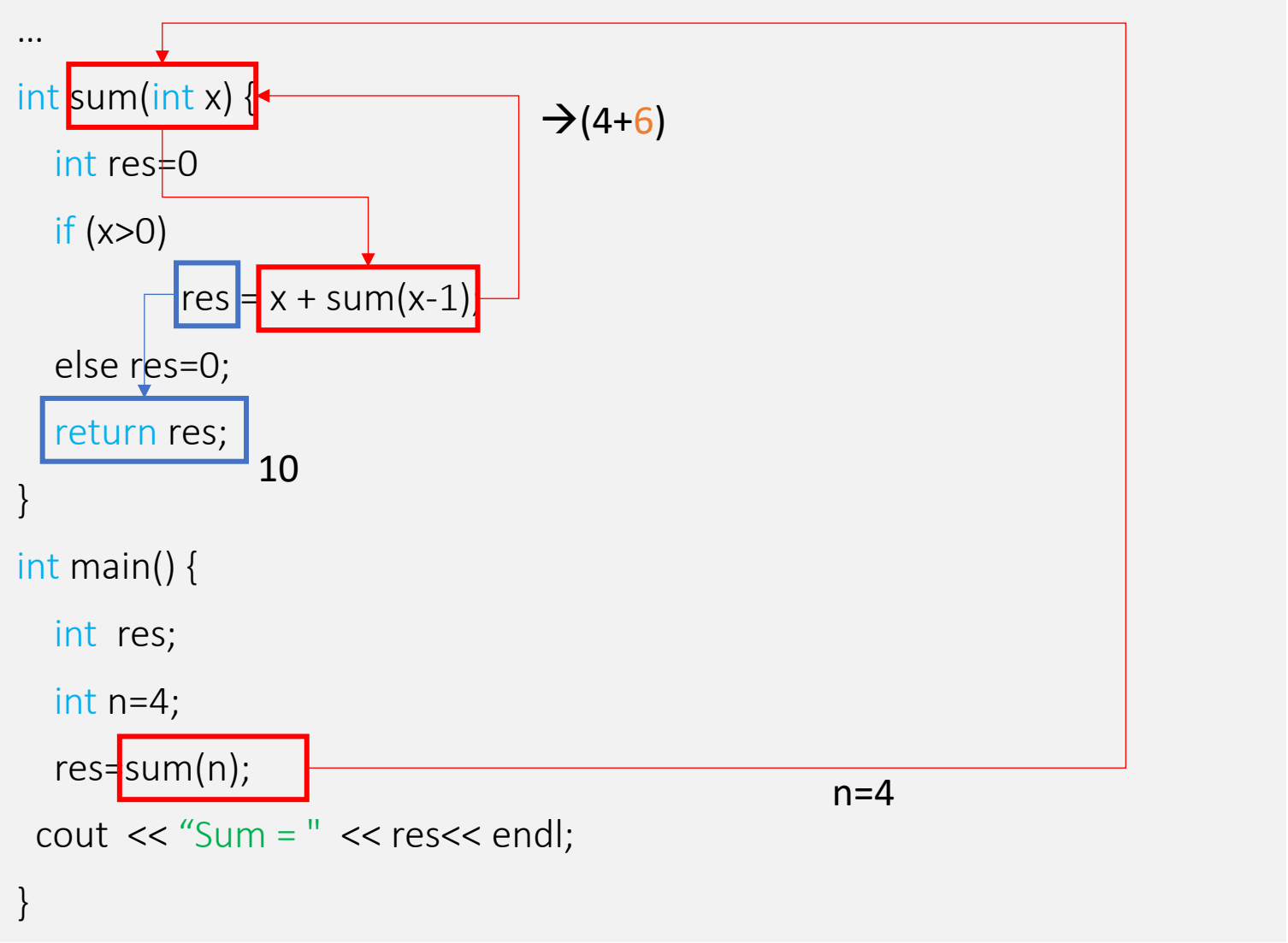
Example of Recursive function: sum

```
...
int sum(int x) {
    int res=0
    if (x>0)
            res = x + sum(x-1);
    else res=0;
    return res;
}
int main() {
    int  res;
    int n=4;
    res=sum(n);
  cout  << "Sum = "  << res<< endl;
}
```

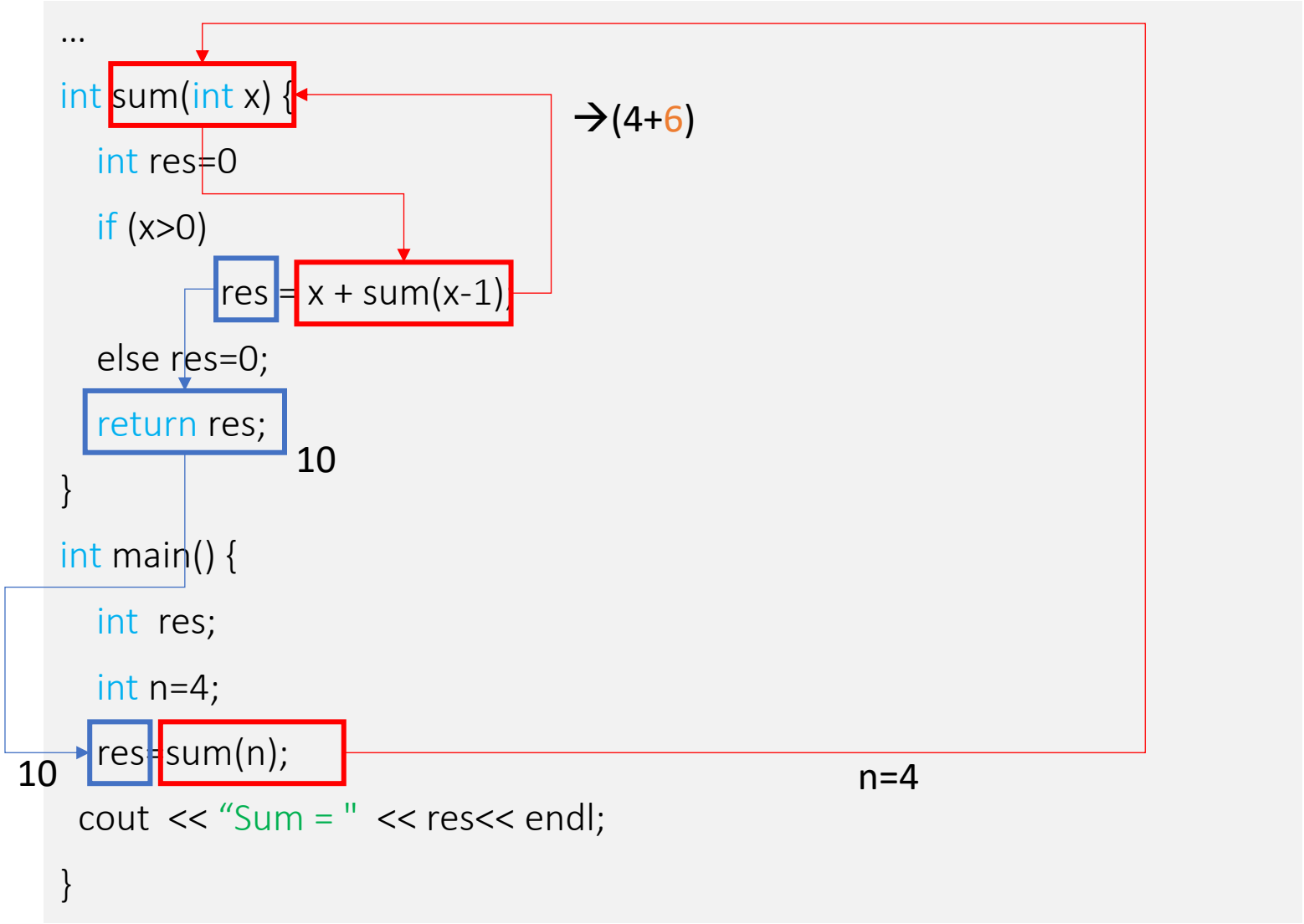
→(4+6)

n=4

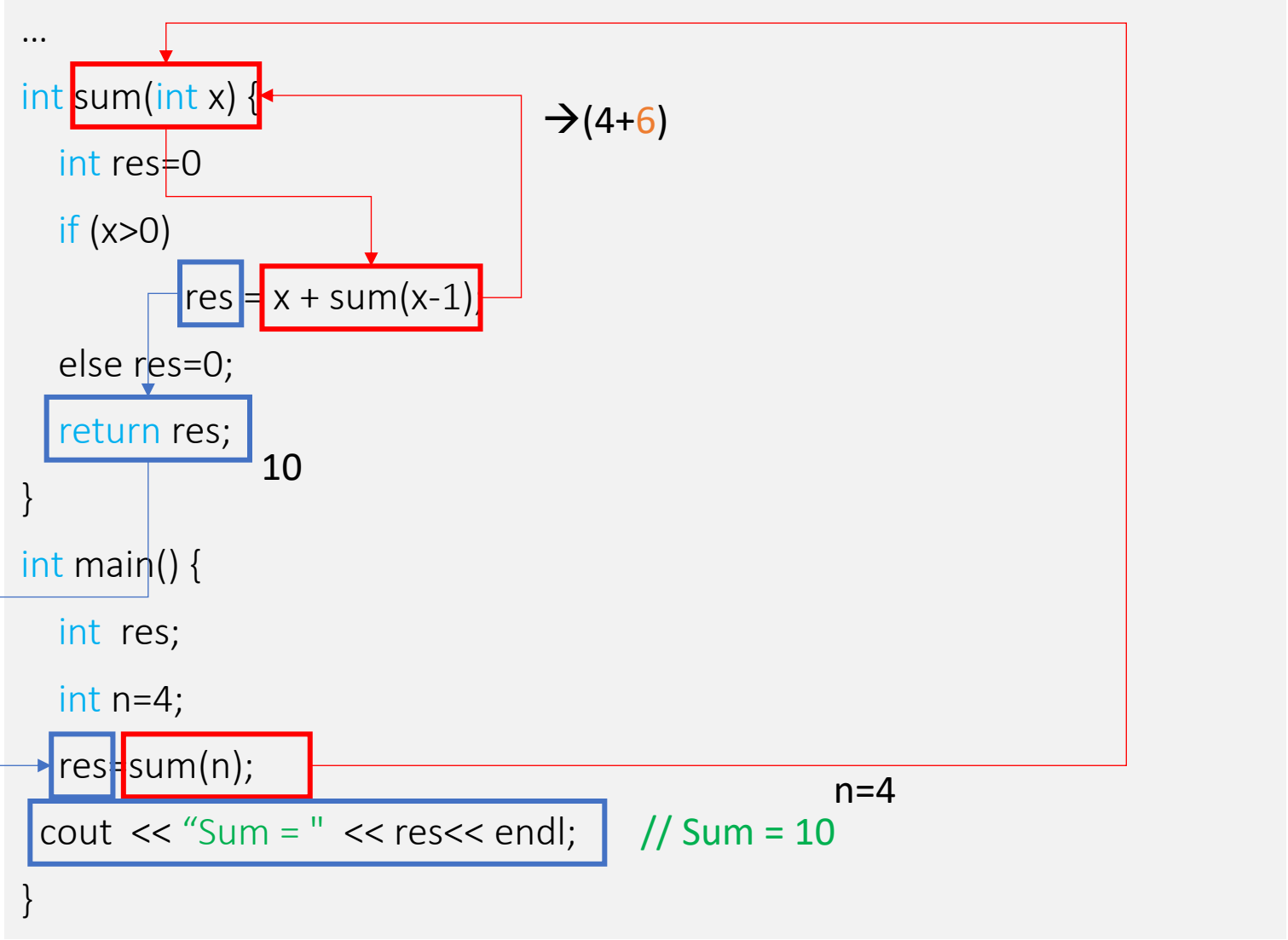
Example of Recursive function: sum



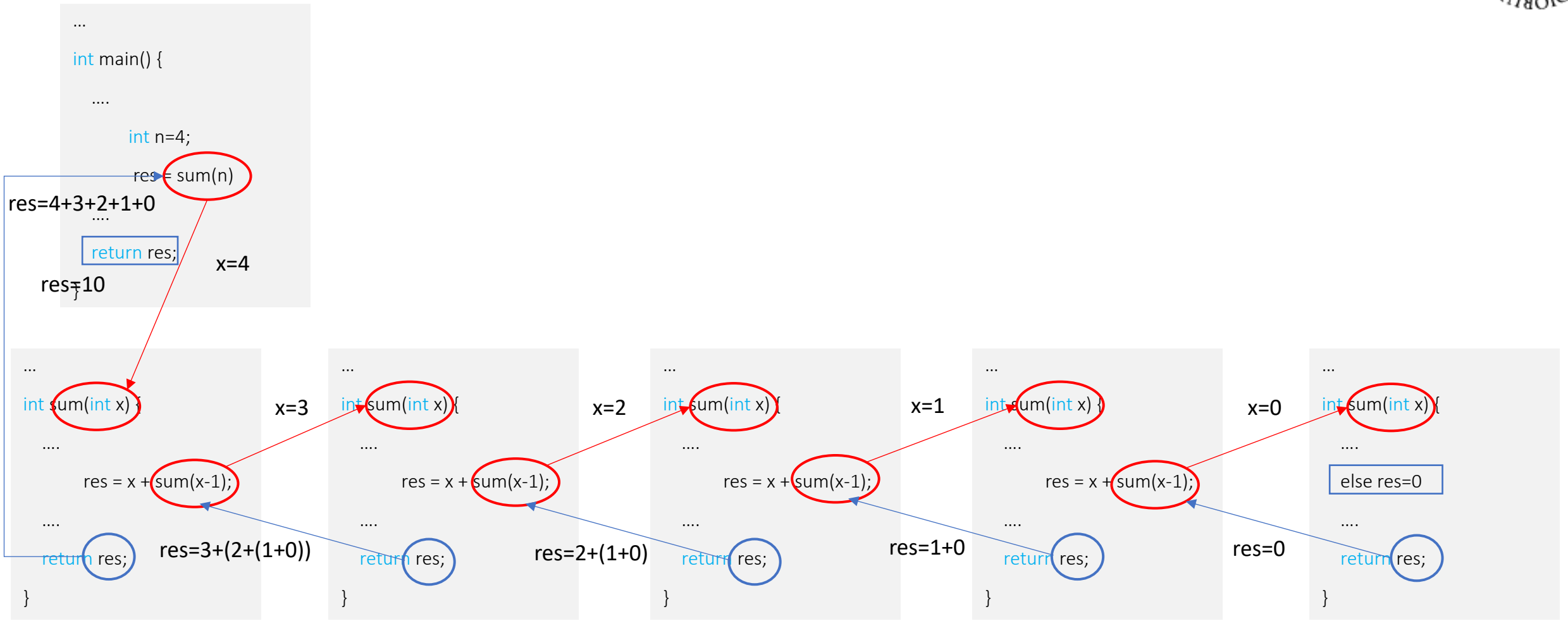
Example of Recursive function: sum



Example of Recursive function: sum



Example of Recursive function: sum (2)



Example of Recursive function: factorial

Iterative (non-recursive) implementation

Recursive implementation

```
...
long long fact(int x) {
  long long fact_var = 1;
  for (int i=1; i<=x; i++)
       fact_var *= i;
  return fact_var;
}

int main() {
   int  num;
  long long factorial;
  cin >> num;
  factorial=fact(num);
  cout  << "Fact = "  << factorial   << endl;
...
```

(num=2)
i=1: fact_var=1*1=1
i=2: fact_var=1*2 =2
fact_var=2

num=2

Example of Recursive function: factorial

Iterative (non-recursive) implementation

```
...
long long fact(int x) {
  long long fact_var = 1;
  for (int i=1; i<=x; i++)
       fact_var *= i;
  return fact_var;
}

int main() {
   int  num;
  long long factorial;
  cin >> num;
  factorial=fact(num);
  cout  << "Fact = "  << factorial   << endl;
...
```

(num=2)
i=1: fact_var=1*1=1
i=2: fact_var=1*2 =2
fact_var=2

num=2

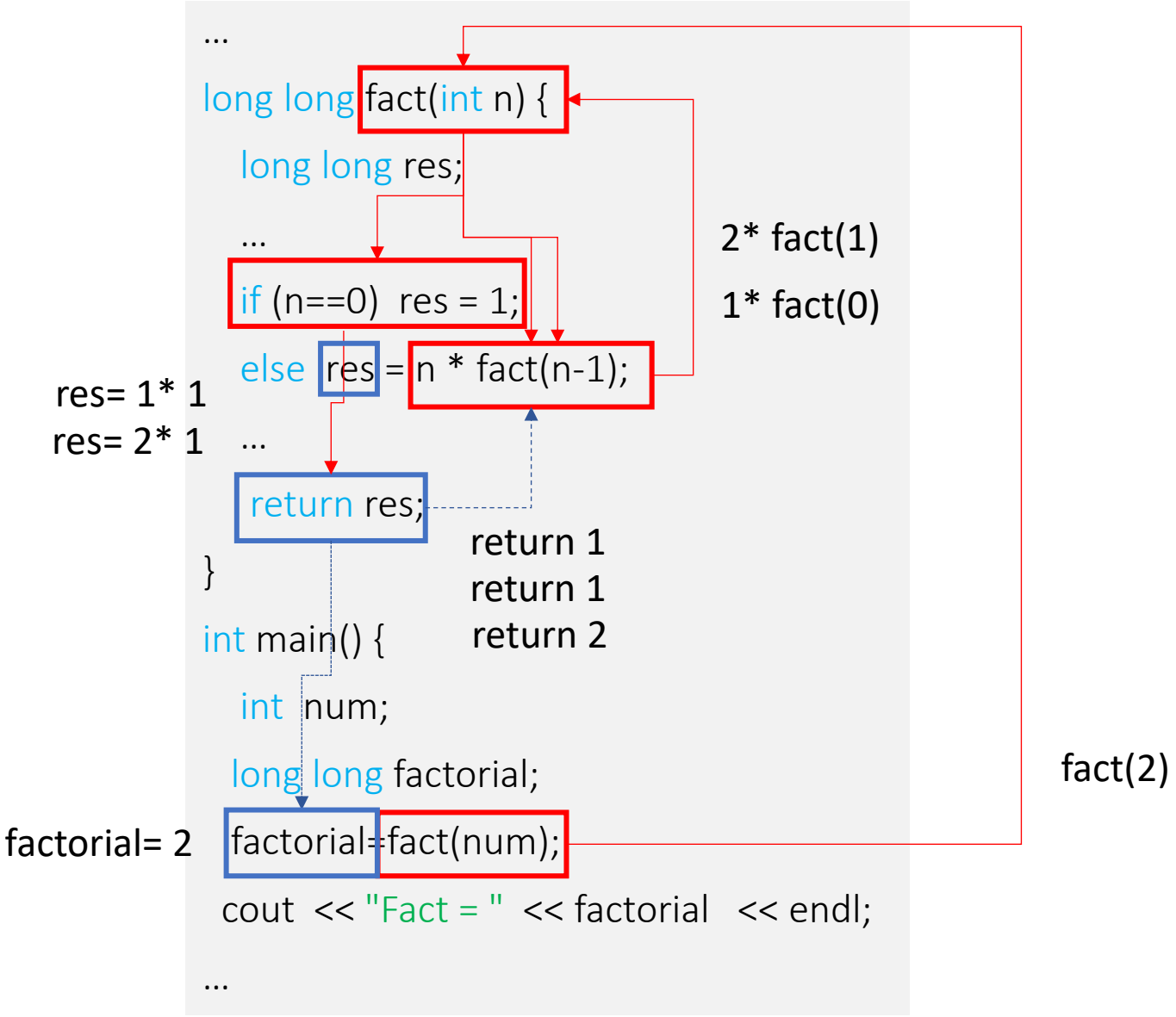
Recursive implementation

```
...
long long fact(int n) {
  long long res;
  ...
  if (n==0)  res = 1;
  else  res = n * fact(n-1);
  ...
   return res;
}
int main() {
  int  num; ...
  long long factorial;
  cin >> num
  factorial=fact(num);
 cout  << "Fact = "  << factorial   << endl;
...
```

(num=2)
→ fact(2)
→ 2 * fact(1)
→ 1* fact(0)
→ n==0, res=1, return 1
→ 1* 1, return 1
→ 2 * 1, return 2
→ res=2

num=2

Example of Recursive function: factorial (2)



Additional readings:
https://www.programiz.com/cpp-programming/recursion

Examples

Example of factorial:

{ ../RECURSIVE_FUNCTIONS/fact_nocomment.cc}

Example of traced calls:

{ ../RECURSIVE_FUNCTIONS/fact.cc}

Example with error (infinite loop):

{ ../RECURSIVE_FUNCTIONS/fact_infloop.cc}

Example of traced stack:

{ ../RECURSIVE_FUNCTIONS/fact_stack.cc}

Example of mutual recursive functions:

{ ../RECURSIVE_FUNCTIONS/evenodd.cc}

Note on the recursion

The recursive realization of a function can require two functions:

- a recursive helper function, with an additional recursion parameter (similar to the loop counter)
- a main function (wrapper) that calls the recursive function with a base value of the recursion parameter
- It is a very frequent situation in the use of arrays

Example of recursive function with wrapper:

{ ./RECURSIVE_FUNCTIONS/printnumbers.cc}

Example of variant 1:

{ ../RECURSIVE_FUNCTIONS/printnumbers1.cc}

Example of variant 2:

{ ../RECURSIVE_FUNCTIONS/printnumbers2.cc}

Example of factorial, with wrapper:

{ ../RECURSIVE_FUNCTIONS/fact_rec1.cc}

Note on the recursion: Example

```
...
long long fact1 (int m, int n) {   // helper: m is the recursion parameter
  //it computes m*m+1*...*n
  if (n==0) return 1;
  if (m==n) return m;
  else return m*fact1(m+1,n);
}
long long fact (int n) {  //wrapper function
   return fact1 (1,n); //base value for the recursion parameter = counter
}
int main() {
  int n;
  cin >> n;
  cout << "factorial(" << n << ") = " << fact(n) << endl;
  return 0;
}
```

- a recursive helper function, with an additional recursion parameter (similar to the loop counter)
- a main function (wrapper) that calls the recursive function with a base value of the recursion parameter

Recursion vs Iteration (1)

- Often, the recursion is more natural, simple and elegant
- Efficiency of critical recursion:
  - Beware of identical calls in different branches! (e.g., Fibonacci)
    → combinatorial explosion risk
  - Stack size depends on the recursion depth
    → significant overhead and memory waste
    → when possible, typically the iteration is more efficient
    → passing “big” objects, it is essential to use passage by reference/pointer

- Many recursive functions can be rewritten in iterative form:
  - tail recursion: a recursive call, operated as the last step
    - E.g., sum, odd / even, ...
  - in general, when it does not involve a "bifurcation"
    - E.g., factorial, Fibonacci, ...
  - The command “g++ -O2” converts from tail-recursive to iterative

From tail-recursion to iteration (*void*)

```
void F(int x,...) {   //x recursion parameter
  if (BaseCase(x,...))  //end condition
     {IstrBase(...);}
  else {
     Istr(...);
     x=update(x,...);
     F(x,...);        // tail-recursion
  }
}
```



```
void F(int x,...) {
  while (!BaseCase(x,...)){
     Istr(...);
     x=update(x,...);
  }
  {IstrBase(...); }
}
```

Example of tail-recursive void function:

{ ../RECURSIVE_FUNCTIONS/printnumbers3.cc}

Example of the corresponding iterative version:

{ ../RECURSIVE_FUNCTIONS/printnumbers3_while.cc}

From tail-recursion to iteration (general case)

```
type F(int x,...) {          //x recursion parameter
 if (BaseCase(x,...))        //end condition
    res = IstrBase(...);
 else {
    Istr(...);
    x=update(x,...);
    res = F(x,...);          //tail-recursion
 }
 return res;
}
```



```
type F(int x,...) {
 while (!BaseCase(x,...)){
    Istr(...);
    x=update(x,...);
 }
 res = IstrBase(...);
 return res;
}
```

Example of tail-recursive function:

{ ../RECURSIVE_FUNCTIONS/sum.cc}

Example of corresponding iterative function:

{ ../RECURSIVE_FUNCTIONS/sum_while.cc}

Example of the compilation from tail-recursive in iterative:

{ ../RECURSIVE_FUNCTIONS/tailrecursive-comp.cc}

From tail-recursion to iteration (example)

```
int sum (int n, int m) {  //n,m recursion parameters
  int res;
  if (m==0)  // base-case for the end condition
    res = n;   //instruction base
  else {
    ++n; --m;  //parameter update
    res = sum (n,m); //tail-recursion
  }
  return res;
}

int main() {
  int n,m;
  cout << "n? m? (>=0):" ;
  cin >> n >> m ;
  cout << "sum(" << n << "," << m << ") = " << sum(n,m) << endl;
return 0;
}
```



```
int sum (int n, int m) {//n,m recursion parameters
  int res;
  while (! m==0) { // (negated) base-case for the end condition
    ++n; --m;  //parameter update
  }
  res = n;  //instruction base
  return res;
}

int main() {
  int n,m;
  cout << "n? m? (>=0):" ;
  cin >> n >> m ;
  cout << "sum(" << n << "," << m << ") = "
       << sum(n,m) << endl;
return 0;
}
```

Recursion vs Iteration (2)

- …
- Sometimes it is not easy to rewrite the recursion in an iterative form
  - non-tail recursive functions, multiple calls
  - E.g., manipulation of expressions

  Example of tail-recursive function:

  { ../RECURSIVE_FUNCTIONS/expression.cc}

- Generally speaking, the conversion of a recursive function in the corresponding iterative function requires the use of a stack